

AUTO CFD WORKSHOP SERIES

AutoCFD5 AIML DrivAerML

Participant Submission Instructions

A reproducible route to prepare native predictions, run the approved evaluator, inspect results, and deliver a verified entry.

PUBLIC TOOLING

Evaluator code, documentation, and immutable profile support.

CONFIDENTIAL ENTRIES

Participant result packages use a private upload-only route.

Version 1.1 / 28 August 2026

<https://github.com/neilashton/autocfd5-aiml-submission>

The complete route, at a glance

You run the official evaluator on your own machine using the pinned DrivAerML dataset. The public repository carries the code and fixed support data; your predictions and results remain under your control until you use the private workshop upload route. The next section lists every official split.

- 01 Clone the evaluator and select the frozen evaluator-v1.1 release.
- 02 Fetch the immutable profile-support bundle and the pinned native test files.
- 03 Create entry.json and native prediction chunks for every selected case.
- 04 Validate the entry, then evaluate one case while developing your export.
- 05 Evaluate the official full split as the minimum common comparison.
- 06 Inspect result.json and selected local HTML profile reports.
- 07 Create and verify one deterministic ZIP, then upload it confidentially.

Public repository does not mean public submissions

Do not commit predictions or results, attach them to an issue, or open a pull request. The organisers provide an AutoCFD Dropbox File Request for confidential upload. The email is only a checksum receipt.

Who is responsible for what

Party	Responsibility
Participant	Export native-order predictions; run, inspect, package, verify, upload, and retain the original ZIP and checksum.
Evaluator	Verify fixed inputs; calculate fields, forces, and profiles; aggregate the full split; write compact, deterministic outputs.
Organisers	Provide the private upload link; verify package identity and split completeness; preserve the received ZIP under the workshop embargo.

What you hand in

- One verified .zip produced by autocfd5-aiml package.
- A short email containing the committee-issued submission ID, filename, and exact SHA-256.
- No raw native prediction files unless the organisers explicitly request a separately hosted immutable artifact.

Questions

Contact neil@neilashton.co.uk or astridwalle@cfdolutions.net, the AutoCFD5 AI/ML TFG organisers.

Use Full as the common baseline

The AutoCFD organising committee asks every participant to evaluate the official Full split as the minimum common comparison. You are welcome to test the other official splits too.

Committee recommendation

Use `split_id = full` for the primary submission. Official split membership is frozen in the evaluator, so participants do not need to repeat its training and validation IDs in `entry.json`.

Official split	Train	Validation	Test	Case set
full	400	34	50	Standard - requested baseline
medium	133	34	50	Standard
scarce	67	34	50	Standard
super_scarce	11	34	50	Standard
geometry	339	48	97	Geometry
high_drag	339	48	97	High drag
low_drag	339	48	97	Low drag
rear_separation	339	48	97	Rear separation

If you use a custom split

The organisers strongly recommend the official splits. If an additional split is not one of those above, give it a new safe `split_id` and include all three ordered run-ID arrays in `entry.json`:

```
{
  "split_id": "my-custom-study",
  "train_case_ids": ["run_1", "run_2"],
  "validation_case_ids": ["run_3"],
  "test_case_ids": ["run_4", "run_5"]
}
```

- Each array must be non-empty, unique, use `run_N` IDs from the pinned dataset, and be disjoint from the other two arrays.
- The evaluator writes `custom-split.json` into the verified result package for the organisers.

```
autocfd5-aiml fetch-data --entry-root my-entry \
  --destination /data/drivaerml --dry-run
```

03 / ENVIRONMENT

Install once; pin everything

Run the scientific calculation on Linux with Python 3.12. The evaluator pins NumPy and VTK because native-file handling and numerical reproducibility are part of the contract.

Required environment

Linux; Python $\geq 3.12, < 3.13$; NumPy 2.2.6; VTK 9.5.2; Git; GitHub CLI (gh); and Hugging Face CLI (hf). A Dockerfile using Python 3.12.13 is included when a clean container is preferable.

Clone and install

```
git clone https://github.com/neilashton/autocfd5-aiml-submission.git
cd autocfd5-aiml-submission
git checkout evaluator-v1.1

python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e '.[test]'
autocfd5-aiml --help
```

Fetch and verify profile support

```
autocfd5-aiml fetch-support --destination support/native-v1
```

The command downloads the **support-v1** release asset, verifies its archive hash, extracts it safely, and verifies **index.json**. It refuses a non-empty destination.

Inspect the native-data download before starting it

```
autocfd5-aiml fetch-data \
  --split-id full \
  --destination /data/drivaerml \
  --dry-run

# Remove --dry-run only when the location and required storage are ready.
autocfd5-aiml fetch-data \
  --split-id full \
  --destination /data/drivaerml
```

The native files are very large. The dry run lists the exact pinned files and sizes. The final dataset root must contain **force_mom_constref_all.csv** and each selected **run_N/** directory in the downloaded layout.

Immutable identities

Dataset	neashton/drivaerml
Dataset revision	7a5c0948ce27be709b1116a3a190f806e7a8f79f
Support release	support-v1
Support ZIP SHA-256	5ebcf744be53016bd158236d1f4af3290ff399b323c0e11a49c37ea9a6c686f6
Support index SHA-256	f47f8c3ed7a56632b0c02a3aec793e4cd823d5d04d5264d00fcd419bf11c0f4f

Declare the entry and exact split

Use the exact submission ID sent to you by the AutoCFD organising committee. Start from the supplied Full-split example and keep its test-case membership and order unchanged.

Create a working entry

```
cp -R examples/entry my-entry
```

entry.json key	Rule
schema	Keep autocfd5-aiml-entry-v1.
schema_version	Keep integer 1.
submission_id	Enter the exact ID sent by the AutoCFD organising committee. It uses lowercase letters, digits, dot, dash, or underscore; at most 80 characters.
method_name	Human-readable method name, 1-200 characters.
contact_email	Email monitored by the participant.
split_id	Keep full for the requested baseline.
test_case_ids	Copy the example exactly. Membership and order are both checked.
prediction_artifact	Optional; only for an organiser-requested private immutable raw artifact.

Minimal shape

```
{
  "schema": "autocfd5-aiml-entry-v1",
  "schema_version": 1,
  "submission_id": "assigned-submission-id",
  "method_name": "Method display name",
  "contact_email": "participant@example.org",
  "split_id": "full",
  "test_case_ids": [ ...copy the complete example array exactly... ]
}
```

The ellipsis above is explanatory and is not valid JSON. Copy examples/entry/entry.json, which contains all 50 IDs.

Validate before producing every case

```
autocfd5-aiml validate-entry my-entry
```

Validation has two layers

validate-entry checks metadata and split declarations. For an official split it requires the frozen test membership and order; for a custom split it requires complete, disjoint, known train, validation, and test IDs. The full evaluation also requires valid surface and volume prediction manifests for every test case.

05 / PREDICTIONS

Export native cells, without reordering

Each case contains separate surface and volume manifests plus one or more compressed NPZ chunks. Raw cell IDs restore native order and must cover the complete native cell range exactly.

```
my-entry/
  entry.json
  cases/
    run_11/
      surface/
        manifest.json
        chunks/chunk-000000.npz
      volume/
        manifest.json
        chunks/chunk-000000.npz
    ...one directory for every case in the selected split...
```

Required NPZ arrays

Association	Array	Dtype	Shape / meaning
Surface	raw_cell_id	int64	(rows,); native polygon ID
Surface	pMeanTrim	float32/64	(rows,); mean pressure
Surface	wallShearStressMeanTrim	float32/64	(rows, 3); wall-shear vector
Volume	raw_cell_id	int64	(rows,); native volume-cell ID
Volume	pMeanTrim	float32/64	(rows,); mean pressure
Volume	UMeanTrim	float32/64	(rows, 3); mean velocity vector

Closed-manifest requirements

- Surface association is `CellData` with support ID `surface_native_cells`.
- The manifest declares case ID, total row count, field component counts, and every chunk.
- Each chunk declares file, SHA-256, row count, and half-open raw-ID range.
- Values must be finite; unknown or missing keys fail validation.
- IDs must cover `[0, total_row_count)` once, with no gaps or duplicates.
- Use at most 1,000,000 rows per chunk unless the organisers publish another limit.

Manifest pattern

```
{
  "format": "drivaerml-native-prediction-chunks-candidate",
  "format_version": 1,
  "artifact_role": "local_evaluator_input_not_official_submission_artifact",
  "case_id": "run_11",
  "support_id": "surface_native_cells",
  "association": "CellData",
  "total_row_count": 7792715,
  "field_components": {"pMeanTrim": 1, "wallShearStressMeanTrim": 3},
  "chunks": [ ...complete chunk records with real hashes... ]
}
```

Do not interpolate onto a convenient grid

The field and force calculation uses the native DrivAerML entities. Preserve native cell identity and export predictions at that resolution. The evaluator verifies each NPZ hash before reading arrays and fails if an input changes during evaluation.

06 / DEVELOPMENT CHECK

Prove one case before the full run

Use a selected split case such as run_11 to catch layout, dtype, hash, ID-coverage, and native-data problems before committing resources to all 50 cases.

Evaluate one case

```
mkdir -p output/dev

autocfd5-aiml evaluate-case \
  --case-id run_11 \
  --dataset-root /data/drivaerml \
  --support-root support/native-v1 \
  --surface-manifest my-entry/cases/run_11/surface/manifest.json \
  --volume-manifest my-entry/cases/run_11/volume/manifest.json \
  --output output/dev/run_11.json
```

A one-case result is diagnostic, not an official result

It contains the complete four field errors, integrated force coefficients, and profile loss statistics for that case. Force and profile R2 require variation across the complete test split, so a single case cannot produce official component or overall scores.

Check the result deliberately

Check	Expected outcome
Native identities	Every boundary, area, and volume part matches its pinned size and SHA-256.
Prediction coverage	Surface and volume IDs are complete, unique, and in the declared ranges.
Numerics	All prediction values and all written JSON numbers are finite.
Forces	Cd, Cl, and pitch moment are integrated from the predicted native surface fields.
Profiles	All 40 series are produced: 20 scored constant-placement and 20 report-only relative-placement series.
Gaps	Unsupported intervals remain separate segments; no line is drawn or integrated across them.

When the output already exists

The evaluator refuses to overwrite a case result. This is intentional. Use a new output filename after changing predictions, so every test result remains attributable to one input set.

Optional clean-container check

```
docker build -t autocfd5-aiml:evaluator-v1.1 .

# Mount entry, native data, support and result locations explicitly.
docker run --rm \
  -v "$PWD/my-entry:/entries/my-entry:ro" \
  -v "/data/drivaerml:/data/drivaerml:ro" \
  -v "$PWD/support/native-v1:/support/native-v1:ro" \
  -v "$PWD/output:/results" \
  autocfd5-aiml:evaluator-v1.1 evaluate-case \
    --case-id run_11 --dataset-root /data/drivaerml \
    --support-root /support/native-v1 \
    --surface-manifest /entries/my-entry/cases/run_11/surface/manifest.json \
    --volume-manifest /entries/my-entry/cases/run_11/volume/manifest.json \
    --output /results/dev/run_11-container.json
```

Run the complete split and inspect it

Official R2 values and the composite score are calculated only after every case in the selected test split has completed. The requested Full baseline contains 50 test cases.

Run all cases

```
autocfd5-aiml evaluate-entry my-entry \
  --dataset-root /data/drivaerml \
  --support-root support/native-v1 \
  --output output/assigned-submission-id \
  --resume
```

If the process is interrupted, repeat the same command with `--resume`. Completed per-case work is reused only after its schema, case ID, and completion state pass validation. After `result.json` exists, that output is complete and immutable; use a new output directory for another model version.

Output layout

```
output/assigned-submission-id/
  result.json           # aggregate metrics and identities
  provenance.json       # runtime and verification record
  cases/run_N.json      # compact result for each test case
  profiles/index.json    # profile prediction index
  profiles/chunk-NNN.json # 40 series per case, eight cases per chunk
  .work/cases/run_N.json # resumable working data; excluded from package
```

Render a local case report

```
autocfd5-aiml report \
  --result-root output/assigned-submission-id \
  --support-root support/native-v1 \
  --case-id run_11 \
  --output output/run_11.html

# Open output/run_11.html in a browser on the same machine.
```

How to read the profile report

Series	Count/case	Role	Display and scoring behaviour
Constant velocity	16	Scored	Raw U/U _{inf} samples; no smoothing; gaps remain separate.
Relative velocity	16	Report only	Same numerical treatment; zero composite weight.
Constant continuous Cp	4	Scored	Shown against physical x; integrated against arc length.
Relative Cp	4	Report only	Two aliases and two moving placements; zero composite weight.

Interpret the plots literally

Adjacent equal velocity values can create visible stair steps because ground truth is retained at its native sampled resolution. Unsupported regions are real gaps and must not be joined. Cp uses physical streamwise x for familiar visual comparison, while its score remains based on the supplied arc-length coordinate.

Nine components, one approved composite

The overall score is on a 0-100 scale. Fields contribute 50%, integrated forces 25%, and constant-placement profiles 25%. Relative-placement profiles remain visible diagnostics.

Group	Component metric	Overall weight	Transform / cap
Fields	Surface pressure relative L2, area weighted	15%	Error score; cap 15%
Fields	Surface wall shear relative L2, area weighted	10%	Error score; cap 20%
Fields	Volume velocity relative L2, equal native cells	15%	Error score; cap 12%
Fields	Volume pressure relative L2, equal native cells	10%	Error score; cap 15%
Forces	Cd R2	15%	Quality score
Forces	Cl R2	5%	Quality score
Forces	Pitch-moment R2	5%	Quality score
Profiles	16 constant velocity profiles: global weighted R2	15%	Quality score
Profiles	4 constant continuous Cp cuts: global weighted R2	10%	Quality score

Transforms

Field-error component

For relative L2 error e and declared cap c : **score** = $\text{clip}(100 \times (1 - e/c), 0, 100)$.

R2 component

For complete-split coefficient of determination R2: **score** = $100 \times \text{clip}(R2, 0, 1)$. The weighted component scores sum to the overall score.

Important scientific details

- Four field errors are complete-case relative L2 percentages, macro-averaged equally across cases.
- Forces are integrated from predicted native surface pressure and wall shear. Pitch truth is $(Cl_f - Cl_r) / 2$.
- Velocity is $\text{magnitude}(U_{\text{MeanTrim}}) / 38.889$. Cp is $2 * p_{\text{MeanTrim}} / 38.889^2$.
- Profile integration follows each scoring coordinate. Every case/profile block is normalized to unit supported length, giving cases and profiles equal weight in global R2.
- Integration stops at every segment boundary. No unsupported interval is bridged and no smoothing is applied.

Why the complete split matters

R2 measures variation across observations. An individual case supplies losses and integrated values, but not the population variation needed for official force R2, profile R2, component scores, or the overall score.

09 / PACKAGE AND DELIVERY

Verify first; upload privately

The packaged result is compact and deterministic. It contains the scientific outputs, profile predictions, entry identity, immutable input hashes, and runtime provenance - not the large raw native prediction chunks by default.

Create the result package

```
autocfd5-aiml package output/assigned-submission-id \  
  --output assigned-submission-id.zip  
  
autocfd5-aiml verify-package assigned-submission-id.zip
```

Replace assigned-submission-id with the exact ID sent by the AutoCFD organising committee. Packaging also writes assigned-submission-id.zip.sha256. Both commands refuse unsafe or inconsistent content. The ZIP is closed by package-manifest.json, which records every member's size and SHA-256.

Private hand-in

- 01 Open the AutoCFD Dropbox File Request.**
- 02 Upload only assigned-submission-id.zip; wait for the upload to complete.**
- 03 Retain the original ZIP and its generated .sha256 file unchanged.**
- 04 Email the organisers the receipt details below. Do not attach the ZIP to email.**
- 05 Keep predictions, results, and reports out of public Git history and issue trackers.**

What the upload-only request protects

Participants can add their own file but cannot browse, download, replace, or compare other participants' submissions. Organisers keep the destination restricted to the processing group until the workshop reveal.

Checksum receipt email

```
Subject: [AutoCFD5 AIML] entry receipt - <submission-id>  
  
Submission ID: <submission-id>  
Method: <method-name>  
Uploaded filename: <submission-id>.zip  
SHA-256: <copy the 64-character value from <submission-id>.zip.sha256>  
Upload completed: <YYYY-MM-DD HH:MM UTC>  
Contact: <contact-email>
```

Optional large prediction artifact

Only when requested, store large native predictions at a private immutable URL and add prediction_artifact to entry.json with exact private_immutable_url, size_bytes, and sha256. The evaluator records the reference but does not copy that artifact into the result ZIP.

Troubleshooting and final checks

Most failures are deliberate fail-closed checks. Correct the input or choose a fresh output location; do not edit a completed result package by hand.

Message or symptom	What it means / what to do
required command is unavailable	Install the named gh or hf CLI and confirm it is on PATH.
support destination is not empty	Use a new empty directory. Support extraction is intentionally non-overwriting.
archive, index, or native source differs	The file is not the pinned object. Re-fetch into a clean location and do not rename, recompress, or edit it.
entry split ID, order or membership differs	For an official split, restore its complete ordered test list. For a custom split, supply complete, disjoint train, validation, and test arrays.
prediction manifest or chunk identity differs	Regenerate manifest sizes and SHA-256 values after writing final NPZ chunks. Do not modify chunks afterward.
raw IDs are missing, repeated, or out of range	Export every native cell exactly once. Do not use solver-local reorderings without mapping back to raw native IDs.
result.json already exists	That output directory is complete. Choose a new output directory for another model version.
interrupted full evaluation	Repeat the identical command with <code>--resume</code> ; validated completed case work is retained.
profile line appears stepped or has a gap	This can be scientifically correct: raw samples are unsmoothed, and unsupported intervals are intentionally disconnected.

Final participant checklist

- [] I used Linux, Python 3.12, and the frozen **evaluator-v1.1** release.
- [] The support and native dataset identities verified automatically.
- [] My `entry.json` uses the submission ID sent by the AutoCFD organising committee and the official Full split.
- [] Every selected case has complete native-order surface and volume predictions.
- [] `evaluate-entry` completed and `result.json` reports the exact split as complete.
- [] I inspected aggregate metrics and at least one local HTML report, including both constant and relative profiles.
- [] `verify-package` reported the final ZIP as valid.
- [] I uploaded the ZIP through the AutoCFD Dropbox File Request and retained the original ZIP plus checksum.
- [] My receipt email contains the exact submission ID, filename, and SHA-256.

Canonical references

Repository: <https://github.com/neilashton/autocfd5-aiml-submission>

Profile support: <https://github.com/neilashton/autocfd5-aiml-submission/releases/tag/support-v1>

DrivAerML dataset: <https://huggingface.co/datasets/neashton/drivaerml>

Submission upload: <https://www.dropbox.com/request/A6cJNTT9egFtYiFICjAi>

Questions: neil@neilashton.co.uk or astridwalle@cfd-solutions.net (AutoCFD5 AI/ML TFG organisers). Do not send an entry through a public repository channel.